Chapter 3

Jump Loop and Call Instruction

Outlines

- Code for using Loop instructions
- Code for using Conditional jump instructions
- Conditions determining conditional jump
- Unconditional long & short jumps
- Calculate target addresses for jumps
- Code for 8051 Subroutines
- Using stack in subroutines
- Crystal frequency vs. machine cycle
- Code programs to generate time delay

Chapter 3:

Section0: Data Move

- **Internal/External Memory**
- **Read/Write**

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,…
- Unconditional Jump: JMP{SJMP,LJMP}

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency

MCS-51 Memory MAP

Add	D
FFFF	xx
..	
0000	xx
8002	xx
8001	xx
8000	xx
7FFF	xx
.	
.	
0010	10
000F	0F
000E	0E
000D	0D
000C	0C
000B	0B
000A	0A
0009	09
0008	08
0007	07
0006	06
0005	05
0004	04
0003	03
0002	02
0001	01
0000	00

← External Memory

Address Index = 2 Byte
Address 0000 to FFFF
ROM = Boot ,OS Area
RAM = Work Area
Port IO

Internal Memory →

Address Index = 1 Byte
Address 00 to FF
SFR [80-FF]
RAM Indirect Access[80-FF]
RAM General Propose[30-7F]
RAM Bit Access[20-2F]
R0-R7 [00-07]

Add	D
FF	xx
..	
85	xx
84	xx
83	xx
82	xx
81	xx
80	xx
7F	
..	
31	
30	
2F	
..	
21	
20	
1F	xx
..	
07	xx
06	xx
05	xx
04	xx
03	xx
02	xx
01	xx
00	xx

Add	D
FF	yy
..	
85	yy
84	yy
83	yy
82	yy
81	yy
80	yy

Read/Write Memory

	Read	Write

Internal Memory 40H	MOV A,40H
	or
	MOV A,@R0
		MOV 40H,A
		or
		MOV @R0,A

External Memory 9000H	MOVX A,@DTPR	MOVX @DPTR,A

Computer Engineering - Suranaree University of Technology

Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,…
- Unconditional Jump: JMP{SJMP,LJMP}

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency

Section1: Loop and jump instruction

Section1.1: Conditional Jump

DJNZ=Decrement and Jump if not zero

Example 3-1

Write a program to
(a) clear ACC, then
(b) add 3 to the accumulator ten times.

Solution:

```
;This program adds value 3 to the ACC ten times

          MOV  A,#0       ;A=0, clear ACC
          MOV  R2,#10     ;load counter R2=10
AGAIN:    ADD  A,#03      ;add 03 to ACC
          DJNZ R2,AGAIN   ;repeat until R2=0(10 times)
          MOV  R5,A       ;save A in R5
```

25,Ex3.1

Loop inside a Loop (Nested Loop)

Example 3-3

Write a program to (a) load the accumulator with the value 55H, and (b) complement the ACC 700 times.

Solution:

Since 700 is larger than 255 (the maximum capacity of any register), we use two registers to hold the count. The following code shows how to use R2 and R3 for the count.

```
          MOV  A,#55H     ;A=55H
          MOV  R3,#10     ;R3=10, the outer loop count
NEXT:     MOV  R2,#70     ;R2=70, the inner loop count
AGAIN:    CPL  A          ;complement A register
          DJNZ R2,AGAIN   ;repeat it 70 times (inner loop)
          DJNZ R3,NEXT
```

In this program, R2 is used to keep the inner loop count. In the instruction "DJNZ R2,AGAIN", whenever R2 becomes 0 it falls through and "DJNZ R3,NEXT" is executed. This instruction forces the CPU to load R2 with the count 70 and the inner loop starts again. This process will continue until R3 becomes zero and the outer loop is finished.

26,Ex3.2

Other Conditional Jump:

Table 3–1: 8051 Conditional Jump Instructions

Instruction	Action
JZ	Jump if A = 0
JNZ	Jump if A ≠ 0
DJNZ	Decrement and jump if register ≠ 0
CJNE A, data	Jump if A ≠ data
CJNE reg, #data	Jump if byte ≠ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

Using Conditional Jump

JZ = Jump if Acc is Zero

```
JZ      Label
```

JNZ = Jump if Acc is not Zero

```
JNZ     Label
```

DJNZ = Decrement and Jump if Not Zero

```
DJNZ  R7,Loop_Label
```

CJNE = Compare and Jump if Not Equal

```
CJNE  A,#data,Label
```

JC = Jump if Carry Set

```
JC      Label
```

JNC = Jump if not Carry Set

```
JNC     Label
```

JB = Jump if Bit Set

```
JB      Px.x,Label
```

JNB = Jump if not Bit Set

```
JNB     20H.x,Label
```

Conditional Jump Example: JNZ,JNC

Example 3-4

Write a program to determine if R5 contains the value 0. If so, put 55H in it.

Solution:

```
        MOV  A,R5       ;copy R5 to A
        JNZ  NEXT       ;jump if A is not zero
        MOV R5,#55H
NEXT:   ...
```

27,Ex3.3

Conditional Jump Example: JNZ,JNC

Example 3-5

Find the sum of the values 79H, F5H, and E2H. Put the sum in registers R0 (low byte) and R5 (high byte).

Solution:

```
        MOV  A,#0       ;clear A(A=0)
        MOV  R5,A       ;clear R5
        ADD  A,#79H     ;A=0+79H=79H
        JNC  N_1        ;if no carry, add next number
        INC  R5         ;if CY=1, increment R5
N_1:    ADD  A,#0F5H    ;A=79+F5=6E and CY=1
        JNC  N_2        ;jump if CY=0
        INC  R5         ;If CY=1 then increment R5(R5=1)
N_2:    ADD  A,#0E2H    ;A=6E+E2=50 and CY=1
        JNC  OVER       ;jump if CY=0
        INC  R5         ;if CY=1, increment 5
OVER:MOV  R0,A          ;Now R0=50H, and R5=02
```

27,Ex3.4

Section1.2: Unconditional Jump Instructions

- **All conditional jumps are short jumps**
  - Target address within -128 to +127 of PC
- **LJMP** (long jump): 3-byte instruction
  - 2-byte target address: 0000 to FFFFH
  - Original 8051 has only 4KB on-chip ROM
- **SJMP** (short jump): 2-byte instruction
  - 1-byte relative address: -128 to +127

Calculating Short Jump Addresses

Example 3-6

Using the following list file, verify the jump forward address calculation.

Line	PC	Opcode		Mnemonic	Operand
01	0000			ORG	0000
02	0000	7800		MOV	R0,#0
03	0002	7455		MOV	A,#55H
04	0004	6003		JZ	NEXT
05	0006	08		INC	R0
06	0007	04	AGAIN:	INC	A
07	0008	04		INC	A
08	0009	2477	NEXT:	ADD	A,#77h
09	000B	5005		JNC	OVER
10	000D	E4		CLR	A
11	000E	F8		MOV	R0,A
12	000F	F9		MOV	R1,A
13	0010	FA		MOV	R2,A
14	0011	FB		MOV	R3,A
15	0012	2B	OVER:	ADD	A,R3
16	0013	50F2		JNC	AGAIN
17	0015	80FE	HERE:	SJMP	HERE
18	0017			END	

29,Ex3.5

Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,…
- Unconditional Jump: JMP{SJMP,LJMP}

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency

Call Instructions

- **LCALL (long call):** 3-byte instruction
  - 2-byte address
  - Target address within 64K-byte range
- **ACALL (absolute call):** 2-byte instruction
  - 11-bit address
  - Target address within 2K-byte range

LCALL (Long Call)

LCALL addr16

C	AC	F0	RS1	RS0	OV		P

Bytes 3
Cycles 2
Encoding

00010010	A_{15}-A_8	A_7-A_0

Operation

```
LCALL
PC = PC + 3
SP = SP + 1
(SP) = PC[7-0]
SP = SP + 1
(SP) = PC[15-8]
PC = addr16
```

Example

```
LCALL SUB1
```


LCALL (Long Call,12 xx xx)

Example 3-8

Write a program to toggle all the bits of port 1 by sending to it the values 55H and AAH continuously. Put a time delay in between each issuing of data to port 1. This program will be used to test the ports of the 8051 in the next chapter.

Solution:

```
        ORG     0
BACK:   MOV     A,#55H      ;load A with 55H
        MOV     P1,A        ;send 55H to port 1
        LCALL   DELAY       ;time delay
        MOV     A,#0AAH     ;load A with AA (in hex)
        MOV     P1,A        ;send AAH to port 1
        LCALL   DELAY
        SJMP    BACK        ;keep doing this indefinitely
;———— this is the delay subroutine
        ORG     300H        ;put time delay at address 300H
DELAY:  MOV     R5,#0FFH    ;R5=255(FF in hex),the counter
AGAIN:  DJNZ    R5,AGAIN    ;stay here until R5 becomes 0
        RET                 ;return to caller (when R5 = 0)
        END                 ;end of asm file
```

30,Ex3.6

CALL Instruction & Role of Stack

Example 3-9

Analyze the stack contents after the execution of the first LCALL in the following.

Solution:

```
0000                    ORG   0
0000 7455   BACK:       MOV   A,#55H   ;load A with 55H
0002 F590               MOV   P1,A     ;send 55H to port 1
0004 120300             LCALL DELAY    ;time delay
0007 74AA               MOV   A,#0AAH;load A with AAH
0009 F590               MOV   P1,A     ;send AAH to port 1
000B 120300             LCALL DELAY
000E 80F0               SJMP BACK      ;keep doing this
0010
0010 ;————————this is the delay subroutine
```


CALL Instruction & Role of Stack

```
0300                    ORG 300H
0300          DELAY:
0300 7DFF               MOV   R5,#0FFH   ;R5=255
0302 DDFE   AGAIN:      DJNZ R5,AGAIN    ;stay here
0304 22                 RET              ;return to caller
0305                    END              ;end of asm file
```

When the first LCALL is executed, the address of the instruction "MOV A,#0AAH" is saved on the stack. Notice that the low byte goes first and the high byte is last. The last instruction of the called subroutine must be a RET instruction which directs the CPU to POP the top bytes of the stack into the PC and resume executing at address 07. The diagram shows the stack frame after the first LCALL.

0A	
09	00
08	07

SP = 09

Using PUSH & POP in Subroutines

Example 3-10

Analyze the stack for the first LCALL instruction in the following program.

```
0000                ORG   0
0000 7455 BACK:     MOV   A,#55H      ;load A with 55H
0002 F590           MOV   P1,A        ;send 55H to port 1
0004 7C99           MOV   R4,#99H
0006 7D67           MOV   R5,#67H
0008 120300         LCALL DELAY       ;time delay
000B 74AA           MOV   A,#0AAH     ;Load A with AA
000D F590           MOV   P1,A        ;send AAH to port 1
000F 120300         LCALL DELAY
0012 80EC           SJMP BACK         ;keep doing this
0014               ;————————this is the delay subroutine
```

32,Ex3.7

Computer Engineering - Suranaree University of Technology

Using PUSH & POP in Subroutines

```
0300                ORG 300H
0300 C004   DELAY:PUSH 4              ;PUSH R4
0302 C005         PUSH 5              ;PUSH R5
0304 7CFF         MOV  R4,#0FFH       ;R4=FFH
0306 7DFF   NEXT: MOV  R5,#0FFH       ;R5=255
0308 DDFE   AGAIN:DJNZ R5,AGAIN
030A DCFA         DJNZ R4,NEXT
030C D005         POP  5            ;POP INTO R5
030E D004         POP  4            ;POP INTO R4
0310 22           RET               ;return to caller
0311              END               ;end of asm file
```

Using PUSH & POP in Subroutines

Solution:

First notice that for the PUSH and POP instructions we must specify the direct address of the register being pushed or popped. Here is the stack frame.

After the first LCALL			After PUSH 4			After PUSH 5		
0B			0B			0B	67	R5
0A			0A	99	R4	0A	99	R4
09	00	PCH	09	00	PCH	09	00	PCH
08	0B	PCL	08	0B	PCL	08	0B	PCL

Calling Subroutines

```
;MAIN program calling subroutines
          ORG  0
MAIN:     LCALL     SUBR_1
          LCALL     SUBR_2
          LCALL     SUBR_3

HERE:     SJMP      HERE
;————————end of MAIN
;
SUBR_1:   ....
          ....
          RET
;————————end of subroutine 1
;
SUBR_2:   ....
          ....
          RET
;————————end of subroutine 2

SUBR_3:   ....
          ....
          RET
;————————end of subroutine 3
          END       ;end of the asm file
```

ACALL (absolute call)

Example 3-11

A developer is using the Atmel AT89C1051 microcontroller chip for a product. This chip has only 1K bytes of on-chip flash ROM. Which of the instructions LCALL and ACALL is most useful in programming this chip?

Solution:
The ACALL instruction is more useful since it is a 2-byte instruction. It saves one byte each time the call instruction is used.

8051 Instruction Set: ACALL

Operation: ACALL
Function: Absolute Call Within 2K Block
Syntax: ACALL *code address*

Instructions	OpCode	Bytes	Flags
ACALL *page0*	0x11	2	None
ACALL *page1*	0x31	2	None
ACALL *page2*	0x51	2	None
ACALL *page3*	0x71	2	None
ACALL *page4*	0x91	2	None
ACALL *page5*	0xB1	2	None
ACALL *page6*	0xD1	2	None
ACALL *page7*	0xF1	2	None

ACALL (absolute call)

ACALL addr11

C	AC	F0	RS1	RS0	OV		P

Bytes 2
Cycles 2
Encoding $A_{10}A_9A_8$10001 | $A_7A_6A_5A_4A_3A_2A_1A_0$

Operation

```
PC = PC + 2
SP = SP + 1
(SP) = PC7-0
SP = SP + 1
(SP) = PC15-8
PC10-0 = A10-0
```

Example

```
ACALL LABEL
```


Programming Efficiently

Example 3-12

Rewrite Example 3-8 as efficiently as you can.

Solution:

```
        ORG   0
        MOV   A,#55H      ;load A with 55H
BACK:   MOV   P1,A        ;issue value in reg A to port 1
        ACALL DELAY       ;time delay
        CPL   A           ;complement reg A
        SJMP  BACK        ;keep doing this indefinitely

;————————this is the delay subroutine
DELAY:
        MOV   R5,#0FFH    ;R5=255(FF in hex),the counter
AGAIN:  DJNZ  R5,AGAIN    ;stay here until R5 becomes 0
        RET               ;return to caller
        END               ;end of asm file
```

Notice in this program that register A is set to 55H. By complementing 55H, we have AAH; and by complementing AAH we have 55H. Why? "01010101" in binary (55H) becomes "10101010" in binary (AAH) when it is complemented; and "10101010" becomes "01010101" if it is complemented.

Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,…
- Unconditional Jump: JMP{SJMP,LJMP}

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency

Time Delay Generation & Calculation

- 1 instruction = $n \times$ machine cycle → $1\,mc = \frac{12}{f}$
- 1 machine cycle = 12 clock cycles → $1\,clk = \frac{1}{f}$

Example 3-13

The following shows crystal frequency for three different 8051-based systems. Find the period of the machine cycle in each case.
(a) 11.0592 MHz (b) 16 MHz (c) 20 MHz

Solution:

(a)11.0592/12 = 921.6 kHz; machine cycle is 1/921.6 kHz = 1.085 μs (microsecond)
(b) 16 MHz/12 = 1.333 MHz; machine cycle (MC) = 1/.1.333 MHz = 0.75 μs
(c) 20 MHz/12 = 1.66 MHz; MC = 1/1.66 MHz = 0.60 μs

34,Ex3.8

Delay Calculation

Example 3-14

For an 8051 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

(a) `MOV R3,#55` (b) `DEC R3` (c) `DJNZ R2,target`
(d) `LJMP` (e) `SJMP` (f) `NOP` (no operation)
(g) `MUL AB`

Solution:

34,Ex3.9

The machine cycle for a system of 11.0592 MHz is 1.085 μs as shown in Example 3-13. Table A-1 in Appendix A shows machine cycles for each of the above instructions. Therefore, we have:

Instruction	*Machine cycles*	*Time to execute*
(a) MOV R3,#55	1	1×1.085 μs = 1.085 μs
(b) DEC R3	1	1×1.085 μs = 1.085 μs
(c) DJNZ R2,target	2	2×1.085 μs = 2.17 μs
(d) LJMP	2	2×1.085 μs = 2.17 μs
(e) SJMP	2	2×1.085 μs = 2.17 μs
(f) NOP	1	1×1.085 μs = 1.085 μs
(g) MUL AB	4	4×1.085 μs = 4.34 μs

MC

Delay Calculation Example

Example 3-15

Find the size of the delay in the following program, if the crystal frequency is 11.0592 MHz.

```
          MOV  A,#55H   01010101
AGAIN:    MOV  P1,A
          ACALL DELAY
          CPL  A        10101010
          SJMP AGAIN
;----Time delay
DELAY:    MOV  R3,#200  1
HERE:     DJNZ R3,HERE  2
          RET           2
```

0
1
⋮
255

1 MC = 1.085 μ

403

403 × 1.085 μ = 437.255 μs

Decliment and Jump If not zero

35,Ex3.10

Delay Calculation Example

Solution:

From Table A-1 in Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine.

			Machine Cycle
DELAY:	MOV	R3,#200	1
HERE:	DJNZ	R3,HERE	2
	RET		1

Therefore, we have a time delay of [(200 × 2) + 1 + 1] × 1.085 μs = 436.17 μs.

Increasing Delay Using NOP

Example 3-16

Find the time delay for the following subroutine, assuming a crystal frequency of 11.0592 MHz.

			Machine Cycle
DELAY:	MOV	R3,#250	1
HERE:	NOP		1
	NOP		1
	NOP		1
	NOP		1
	DJNZ	R3,HERE	2
	RET		1+1

250

การวนลูป 1+1500+2=1503

การวน = 1500

1+6x250+2

Solution:

The time delay inside the HERE loop is [250 (1+1+1+1+2)] × 1.085 μs = 1500 × 1.085 μs = 1627.5 μs. Adding the two instructions outside the loop we have 1627.5 μs + 2 × 1.085 μ s = 1629.67 μs.

35,Ex3.11

Delay Calculation Example

Example 3-17

For a machine cycle of 1.085 μs, find the time delay in the following subroutine.

```
DELAY:                          Machine Cycle
          MOV   R2,#200              1
AGAIN:    MOV   R3,#250              1
HERE:     NOP                        1
          NOP                        1
          DJNZ  R3,HERE              2
          DJNZ  R2,AGAIN             2
          RET                        2
```

200

250

$1+(1+4\times250+2)200+2$

200603 MC

36,Ex3.12

For the HERE loop, we have (4 × 250) ×1.085 μs = 1085 μs. The AGAIN loop repeats the HERE loop 200 times; therefore, we have 200 × 1085 μs = 217000, if we do not include the overhead. However, the instructions "MOV R3,#250" and "DJNZ R2,AGAIN" at the beginning and end of the AGAIN loop add (3 × 200 × 1.085 μs) = 651 μs to the time delay. As a result we have 217000 + 651 = 217651 μs = 217.651 milliseconds for total time delay associated with the above DELAY subroutine. Notice that in the case of a nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.

Delay Calculation for other versions of 8(

-The Crystal Frequency

11.059MHz, 18.432MHz, 12MHz

-The 8051 Design {Clock per Instruction}

Chip/Maker	Clocks per Machine Cycle
AT89C51 Atmel	12
P89C54X2 Philips RD2	6
DS5000 Dallas Semi	4
DS89C420/30/40/50 Dallas Semi	1

ใช้ปกติ

Delay Calculation Example

Example 3-18

From Table 3-2, find the period of the machine cycle (MC) in each case if XTAL = 11.0592 MHz, and discuss the impact on performance.
(a) AT89C51 (b) P89C54X2 (c) DS5000 (d) DS89C4x0

Solution:

(a) 11.0592 MHz/12 = 921.6 kHz; MC is 1/921.6 kHz = 1.085 μs (microsecond) = 1085 ns
(b) 11.0592 MHz/6 = 1.8432 MHz; MC is 1/1.8432 MHz = 0.5425 μs = 542 ns
(c) 11.0592 MHz/4 = 2.7648 MHz; MC is 1/2.7648 MHz = 0.36 μs = 360 ns
(d) 11.0592 MHz/1 = 11.0592 MHz; MC is 1/11.0592 MHz = 0.0904 μs = 90 ns

This means that if we connect an AT89C51 and a DS89C4x0 to a crystal of the same frequency we get approximately 9 to 10 times performance boost for the DS89C4x0 chip over the AT89C51. See Example 3-20.

Delay Calculation for DS89C4x0

Instruction	8051	DS89C4x0
MOV R3,#value	1	2
DEC Rx	1	1
DJNZ	2	4
LJMP	2	3
SJMP	2	3
NOP	1	1
MUL AB	4	9

Table 3–3: Comparison of 8051 and DS89C4x0 Machine Cycles

Delay Calculation Example

Example 3-19

For an AT8051 and DS89C420/30/40/50 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

(a) `MOV R3,#55` (b) `DEC R3` (c) `DJNZ R2,target`
(d) `LJMP` (e) `SJMP` (f) `NOP` (no operation)
(g) `MUL AB`

Solution:
The machine cycle time for the AT8951 and DS89C420/30 was shown in Example 3-18. Table 3-3 shows machine cycles for each of the above instructions. Therefore, we have:

Instruction	*AT8051*	*DS89C420/30/40/50*
(a) MOV R3,#55	1×1085 ns = 1085 ns	2×90 ns = 180 ns
(b) DEC R3	1×1085 ns = 1085 ns	1×90 ns = 90 ns
(c) DJNZ R2,..	2×1085 ns = 2170 ns	4×90 ns = 360 ns
(d) LJMP	2×1085 ns = 2170 ns	3×90 ns = 270 ns
(e) SJMP	2×1085 ns = 2170 ns	3×90 ns = 270 ns
(f) NOP	1×1085 ns = 1085 ns	1×90 ns = 90 ns
(g) MUL AB	4×1085 ns = 4340 ns	9×90 ns = 810 ns

Delay Calculation Example

Example 3-20

Find the time delay for the loop section of the following subroutine if it is run on a DS89C420/30 chip, assuming a crystal frequency of 11.0592 MHz.

```
                                DS89C420/30 Machine Cycle
DELAY:    MOV  R3,#250

HERE:     NOP                   1
          NOP                   1
          NOP                   1
          NOP                   1
          DJNZ R3,HERE          4

          RET
```

Solution:

The time delay inside the HERE loop is $[250(1 + 1 + 1 + 1 + 4)] \times 90$ ns $= 2000 \times 90$ ns $= 180$ µs. Comparing this with Example 3-16, we see DS89C4x0 is about 9 times faster. (1627 µs / 180 µs = 9)

Delay Calculation Example

Example 3-21

Write a program to toggle all the bits of P1 every 200 ms. Assume that the crystal frequency is 11.0592 MHz, and that the system is using the AT89C51.

Solution:

```
;Tested for AT89C51 of 11.0592 MHz.

          MOV   A,#55H
AGAIN:    MOV   P1,A
          ACALL DELAY
          CPL   A
          SJMP  AGAIN

;----Time delay
DELAY:    MOV   R5,#2
HERE1:    MOV   R4,#180
HERE2:    MOV   R3,#255
HERE3:    DJNZ  R3,HERE3
          DJNZ  R4,HERE2
          DJNZ  R5,HERE1
          RET
```

$2 \times 180 \times 255 \times 2$ MC $\times$ 1.085 μs = 199,206 μs

Computer Engineering - Suranaree University of Technology

Delay Calculation Example

Example 3-22

Write a program to toggle all the bits of P1 every 200 ms. Assume crystal frequency is 11.0592 MHz and the system is using DS89C420/30/40/50.

Solution:

```
;Tested for DS89C420 of 11.0592 MHz.
          MOV   A,#55H
AGAIN:    MOV   P1,A
          ACALL DELAY_200m
          CPL   A
          SJMP  AGAIN


;----Time delay
DELAY_200m:
          MOV   R5,#9
HERE1:    MOV   R4,#242
HERE2:    MOV   R3,#255
HERE3:    DJNZ  R3,HERE3
          DJNZ  R4,HERE2
          DJNZ  R5,HERE1
          RET
```

Delay $9 \times 242 \times 255 \times 4$ MC $\times$ 90 ns = 199,940 μs

Use an oscilloscope to measure the system square wave period to verify delay.

SJMP to itself suing $ sign

```
HERE:   SJMP    HERE
        Or
        SJMP    $
```


Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,…
- Unconditional Jump: JMP{SJMP,LJMP}

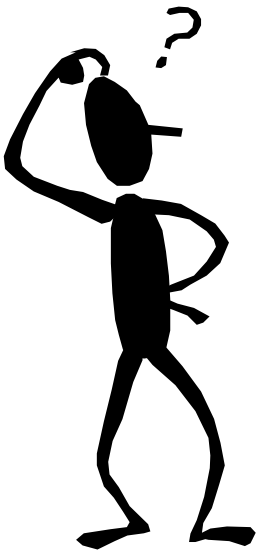
Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency

จบการบรรยาย



15:30

รหัส:_____

ชื่อ-สกุล:_____

1. จงคำนวณหาค่า time delay ของโปรแกรมย่อย DELAY ต่อไปนี้ ว่ามีค่าเท่ากับเท่าใด ถ้า 8051 ใช้เบอร์ AT89C51-16PC

```
DELAY:   MOV   R2,#150
AGAIN:   MOV   R4,#250
HERE:    DJNZ  R4,HERE
         DJNZ  R2,AGAIN
         RET
```

2. คำนวณหาค่า time delay ของโปรแกรมย่อย DELAY ต่อไปนี้ ว่ามีค่าเท่ากับเท่าใด ถ้า crystal frequency เท่ากับ 20 MHz และ 8051 ใช้เบอร์ P89C54X2 ของฟิลลิปส์

```
DELAY:   MOV   R1,#50
SUT3:    MOV   R2,#100
SUT2:    MOV   R3,#250
SUT1:    DJNZ  R3,SUT1
         DJNZ  R2,SUT2
         DJNZ  R1,SUT3
         RET
```

3. ให้เขียนโปรแกรมวนลูปเพื่อเติมค่าคงที่ 00 ในหน่วยความจำภายนอกตั้งแต่ตำแหน่ง 8000H ถึงตำแหน่ง 8FFFH

4. ให้นักศึกษาเขียนโปรแกรมเพื่อตรวจสอบว่าค่าข้อมูลที่อยู่ในหน่วยความจำภายในตำแหน่ง 40H-7FH ว่ามีค่าเท่ากับค่าที่เก็บในรีจีสเตอร์ B ก็ค่าโดยเก็บค่าที่นับได้ในหน่วยความจำภายนอกตำแหน่ง 9000H

5. ให้นักศึกษาเขียนโปรแกรมย้ายข้อมูลในหน่วยความจำ 30H ถึง 7FH ไปไว้ที่หน่วยความจำภายนอกตำแหน่งที่ 9000H เป็นต้นไป โดยกำหนดเงื่อนไขการย้าย คือ _____